

Smaller and Shallower XOR Circuits for AES MixColumns: 97 Gates at Depth 3; 92 at Depth 4; 89 at Depth 5

Joe Bachir
Independent researcher
joebachir20@gmail.com

July 2026

Abstract

We present three implementations of the AES MixColumns linear transformation as circuits of 2-input XOR gates over $\text{GF}(2)$ that improve the entire published depth-count Pareto frontier: a **97-gate** circuit at depth **3** (the known minimum depth), improving the 99-gate record of Shi, Feng, and Xu (ToSC 2023); a **92-gate** circuit at depth **4**, improving the 97-gate point of Osvik and Canright (ePrint 2024/1076); and an **89-gate** circuit at depth **5** — the smallest XOR count we are aware of at any depth — improving Osvik and Canright’s 94-gate depth-5 point by five gates and the smallest published count in any comparable model (91, Lin, Xiang, Zeng, and Zhang, CT-RSA 2021, s-XOR) by two. The circuits were found by a stochastic local search over a value-set formulation of the shortest-linear-program problem, built around moves that are individually neutral or worse but globally productive; the method is fully specified, and both it and the circuits are published as machine-checkable artifacts with dependency-free Python verifiers and reproducers. We make no optimality claim on gate counts.

1 Model and specification

A circuit is an ordered list of 2-input XOR gates. Signals $s_0, \dots, s_{31}$ are the input bits; gate k produces signal $s_{32+k} = s_a \oplus s_b$ with a, b both strictly smaller than $32 + k$. Cost is the number of gates; depth counts gates on the longest input-to-output path, with inputs at depth 0.

The bit/byte convention is: the AES state column is four bytes a_0, a_1, a_2, a_3 ; bit index $i \in \{0, \dots, 31\}$ denotes bit $(i \bmod 8)$ of byte $\lfloor i/8 \rfloor$, least-significant-bit first within each byte. MixColumns maps the input column to the output column with

$$\text{out}[c] = 2 \cdot a_c \oplus 3 \cdot a_{(c+1) \bmod 4} \oplus a_{(c+2) \bmod 4} \oplus a_{(c+3) \bmod 4},$$

with multiplication in $\text{GF}(2^8)$ modulo $x^8 + x^4 + x^3 + x + 1$ (0x11b), following NIST FIPS 197-upd1, Section 5.1.3, Eq. 5.6 [1]. Because MixColumns is linear over $\text{GF}(2)$, agreement on the 32 unit-input vectors is a complete correctness check. Gate counts and depths are invariant under input/output bit relabeling, so the comparisons below do not depend on convention choices.

The s-XOR (in-place) cost model is comparable to this one: an s-XOR program is a sequence of instructions $x_i \leftarrow x_i \oplus x_j$, and unrolling the overwritten registers (SSA-style) turns a k -instruction program into exactly k free-standing 2-input XOR gates. Published s-XOR counts therefore bound the published state of the art in our model. Results in other cost models — multi-input XOR gates, technology-mapped gate-equivalent area, in-place quantum CNOT circuits — are not comparable and are not claimed against.

2 Results

Depth	Published best	Model	This work	Improvement
3	99 [2]	XOR2	97	−2 (depth 3 is the minimum depth)
4	97 [3]	XOR2	92	−5
5	94 [3]	XOR2	89	−5; smallest count at any depth
6	92 [4, 5]	XOR2 / s-XOR	—	dominated by 89 @ 5
7	91 [6, 7]	s-XOR	—	dominated by 89 @ 5

Table 1: The published depth–count Pareto frontier for AES MixColumns in comparable models, and this work. Depth-3 lineage: $105 \rightarrow 103 \rightarrow 102 \rightarrow 99$ (see [2], Table 3). Full circuit listings are in Appendices A–C.

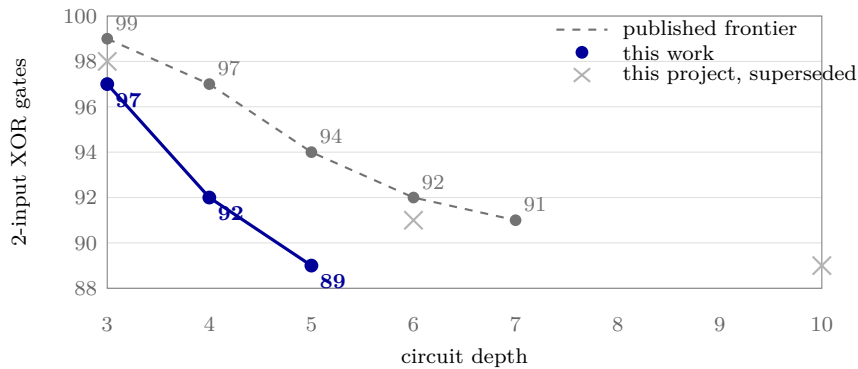


Figure 1: The published depth–count Pareto frontier for AES MixColumns in comparable models (dashed) and this work (solid).

Depth 3 is the known minimum depth for AES MixColumns (stated e.g. in [2]: an output depending on w inputs needs depth at least $\lceil \log_2 w \rceil$, and MixColumns has outputs of weight 7), so the 97-gate circuit’s contribution is the count at that depth. The 89-gate depth-5 circuit Pareto-dominates every published point: fewer gates than the smallest published count at any depth, at lower depth than every published circuit of fewer than 94 gates.

3 Method

The circuits were found by stochastic local search over a *value-set* formulation of the shortest-linear-program problem: a circuit is represented as a set of $\text{GF}(2)$ masks over the 32 inputs, valid iff every mask is the XOR of two already-available masks (realizability) and all 32 output masks are present; the gate count is the set size.

The starting observation is that in the published record circuits every gate is locally efficient — the signature of greedy construction — which suggests such circuits sit at local optima, and that smaller circuits are separated from them by locally *suboptimal* intermediate steps. The search is therefore built around moves that are individually neutral or worse but globally productive:

- **remove-1**: drop a non-output mask if the set remains realizable;
- **neutral swap (plateau walk)**: remove one mask and add one repair mask that restores realizability — size unchanged — walking the plateau of equal-size circuits until some mask becomes jointly redundant;

- **remove-2-add-1 (hub move):** the compound move that nets -1 when single removals are exhausted;
- **destroy-and-rebuild:** remove k masks and greedily re-synthesize from a pool of candidate masks, accepting rebuilds that are no larger, or slightly larger with fixed probability (the uphill steps);
- **depth-aware reconstruction:** a fixpoint computation of each mask’s minimum build depth over the set, used to reject depth-cap violations and to emit gate lists in depth order.

The core loop, for the seedable engines, is:

```

S := value-set of a starting circuit (or of the naive per-output XOR trees)
best := S
repeat:
  with probability p_hub: remove two non-output masks from S
  otherwise:             remove one non-output mask from S
  if S is no longer realizable:
    search (bounded, randomized) for one repair mask w with
      S ∪ {w} realizable; if none, undo and continue
  S := trim(S) // drop masks the 32 outputs no longer need
  if depth-capped and min-build-depth(S) > cap: undo and continue
  if |S| < |best|, or |S| = |best| at strictly lower depth:
    verify against the GF(28) specification; if correct, best := S

```

With the published reproducer this loop, in dependency-free single-core Python, cuts the published 92-gate circuit of [5] to 91 gates in seconds and a 90-gate seed to 89 in about two minutes; the from-scratch depth-3 constructor below reaches 97 gates in one to three minutes.

The depth-3 circuit comes from a from-scratch constructor in the same spirit: each output is modeled as $A \oplus B$ where A, B partition its bits into depth- ≤ 2 parts shared across outputs and recount-costed, optimized by simulated annealing with greedy descent and iterated local search; depth ≤ 3 holds by construction. Deeper records come from a ladder of parallel workers: the depth-3 circuit seeds a depth-4 search, whose best seeds depth 5, and so on, while a coordinator continuously re-offers every worker the best circuit feasible at its depth cap. Workers accept candidates that have fewer gates *or equal gates at strictly lower depth* (a Pareto tie-break; the 89-gate circuit moved from depth 6 to depth 5 through exactly this rule). Every candidate is verified against MixColumns rebuilt from GF(2⁸) before it is accepted (verify-before-claim).

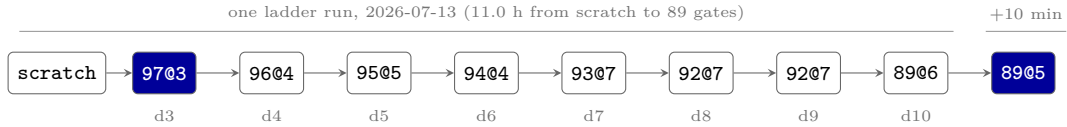


Figure 2: The fully logged lineage of the 89-gate depth-5 circuit (gates@depth per ladder rung): from scratch to 89 gates in one run, then one Pareto-tie-break reroute to depth 5. Every arrow is a seeded search step with a timestamped log line and a verified intermediate circuit in the method repository’s run archives.

The records are replicable with the published configurations: the from-scratch 97 at depth 3 in about one to three minutes on a single core; the 89 at depth 5 in about ten to fifteen minutes from the shipped 89-at-depth-6 seed; and the full from-scratch ladder in hours.

Implementation and execution of the search campaigns used LLM coding agents as programming tools, directed by the author; the moves and acceptance rules above were designed

by the author. The method as published is fully specified and reproducible with the accompanying dependency-free Python code, with no AI system in the loop: the from-scratch 97-gate depth-3 circuit reproduces in minutes on a single core, and the method repository documents the full lineage of every record, from scratch to 89 gates, with the exact code, logs, and every intermediate verified circuit of the runs that produced them.

4 Verification and artifacts

The circuits are static, machine-checkable artifacts, published with verifiers at

<https://github.com/Joe-b-20/aes-mixcolumns-xor-circuits>

and archived permanently at DOI 10.5281/zenodo.21299092; the method, pipeline, and run evidence are published at

<https://github.com/Joe-b-20/slp-plateau-search>

Every claim in this note is decidable from the note alone, with nothing external trusted: the appendices list every gate of every circuit, Section 1 defines the target map, and because MixColumns is linear, replaying a listing on the 32×32 GF(2) unit-input vectors and comparing against the matrix derived from Eq. 5.6 of [1] is a complete correctness check — roughly thirty lines of code in any language. Gate counts and depths are read directly off the listings.

The published artifacts automate this. Each circuit is verified by two separately implemented, dependency-free Python paths — `verify.py`, which rebuilds the 32×32 GF(2) target map from the GF(2⁸) definition (the verifier *is* the specification), and a clean-room verifier with deterministic random trials and adversarial rejection tests — plus Verilog testbenches under Icarus Verilog, all run by continuous integration. Each appendix states its circuit’s canonical SHA-256, computed over the UTF-8 bytes of the compact JSON string `{"inputCount":32,"gates":[...]}` and pinned in the repository’s integrity metadata.

5 Scope and claims

- **Not optimality.** Minimum 2-input-XOR circuit size (the Shortest Linear Program problem) is NP-hard; we do not prove any of these counts minimal. We claim only that they are the smallest we have found or seen published as of July 2026.
- **Comparisons.** All counts are for the exact model and convention of Section 1; the baselines [2, 3, 4, 5, 6, 7] were source-checked, and a source-by-source audit (with the non-comparable cost models explicitly listed) is maintained in the artifact repository’s `PRIOR_ART.md`.
- **Earlier results.** This note supersedes the project’s earlier circuits (89 gates at depth 10; 98 at depth 3; 91 at depth 6), which remain in the artifact repository for the archival record.

References

- [1] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, NIST FIPS 197-upd1, May 2023. <https://doi.org/10.6028/NIST.FIPS.197-upd1>
- [2] H. Shi, X. Feng, and S. Xu, *A Framework with Improved Heuristics to Optimize Low-Latency Implementations of Linear Layers*, IACR Transactions on Symmetric Cryptology, 2023(4):489–510, 2023. <https://doi.org/10.46586/tosc.v2023.i4.489-510>

- [3] D. A. Osvik and D. Canright, *A More Compact AES, and More*, IACR Cryptology ePrint Archive, Report 2024/1076, 2024. <https://eprint.iacr.org/2024/1076>
- [4] A. Maximov, *AES MixColumn with 92 XOR gates*, IACR Cryptology ePrint Archive, Report 2019/833, 2019. <https://eprint.iacr.org/2019/833>
- [5] Z. Xiang, X. Zeng, D. Lin, Z. Bao, and S. Zhang, *Optimizing Implementations of Linear Layers*, IACR Transactions on Symmetric Cryptology, 2020(2):120–145, 2020. <https://doi.org/10.13154/tosc.v2020.i2.120-145>
- [6] D. Lin, Z. Xiang, X. Zeng, and S. Zhang, *A Framework to Optimize Implementations of Matrices*, Topics in Cryptology – CT-RSA 2021, LNCS 12704, Springer, 2021. https://doi.org/10.1007/978-3-030-75539-3_25
- [7] Y. Yuan, W. Wu, T. Shi, L. Zhang, and Y. Zhang, *A Framework to Improve the Implementations of Linear Layers*, IACR Transactions on Symmetric Cryptology, 2024(2):322–347, 2024. <https://doi.org/10.46586/tosc.v2024.i2.322-347>

A Circuit mixcolumns_97gates_depth3 (97 gates, depth 3)

Signals $s_0, \dots, s_{31}$ are the inputs; gate k produces s_{32+k} . Canonical gate-list SHA-256 (see Section 4):

e4c4e678c4b7c22d889c6e060b43859c111b82b953d6d9b43c49c079c0545b56

s32 = s0 $\oplus$ s8	s65 = s36 $\oplus$ s43	s98 = s70 $\oplus$ s60
s33 = s5 $\oplus$ s13	s66 = s43 $\oplus$ s19	s99 = s74 $\oplus$ s77
s34 = s6 $\oplus$ s14	s67 = s32 $\oplus$ s24	s100 = s78 $\oplus$ s61
s35 = s7 $\oplus$ s15	s68 = s36 $\oplus$ s24	s101 = s63 $\oplus$ s83
s36 = s8 $\oplus$ s16	s69 = s44 $\oplus$ s37	s102 = s87 $\oplus$ s85
s37 = s9 $\oplus$ s17	s70 = s32 $\oplus$ s46	s103 = s59 $\oplus$ s56
s38 = s10 $\oplus$ s18	s71 = s46 $\oplus$ s10	s104 = s93 $\oplus$ s15
s39 = s11 $\oplus$ s19	s72 = s46 $\oplus$ s16	s105 = s65 $\oplus$ s67
s40 = s11 $\oplus$ s20	s73 = s45 $\oplus$ s17	s106 = s65 $\oplus$ s73
s41 = s12 $\oplus$ s20	s74 = s45 $\oplus$ s38	s107 = s75 $\oplus$ s18
s42 = s6 $\oplus$ s22	s75 = s47 $\oplus$ s37	s108 = s80 $\oplus$ s66
s43 = s15 $\oplus$ s23	s76 = s47 $\oplus$ s39	s109 = s84 $\oplus$ s66
s44 = s1 $\oplus$ s24	s77 = s46 $\oplus$ s26	s110 = s62 $\oplus$ s55
s45 = s1 $\oplus$ s25	s78 = s49 $\oplus$ s35	s111 = s86 $\oplus$ s89
s46 = s9 $\oplus$ s25	s79 = s50 $\oplus$ s18	s112 = s93 $\oplus$ s64
s47 = s2 $\oplus$ s26	s80 = s50 $\oplus$ s38	s113 = s67 $\oplus$ s58
s48 = s11 $\oplus$ s26	s81 = s51 $\oplus$ s19	s114 = s94 $\oplus$ s72
s49 = s2 $\oplus$ s27	s82 = s51 $\oplus$ s41	s115 = s75 $\oplus$ s71
s50 = s3 $\oplus$ s27	s83 = s53 $\oplus$ s3	s116 = s95 $\oplus$ s79
s51 = s4 $\oplus$ s27	s84 = s52 $\oplus$ s40	s117 = s96 $\oplus$ s81
s52 = s4 $\oplus$ s28	s85 = s52 $\oplus$ s21	s118 = s87 $\oplus$ s62
s53 = s12 $\oplus$ s28	s86 = s42 $\oplus$ s54	s119 = s89 $\oplus$ s34
s54 = s13 $\oplus$ s29	s87 = s53 $\oplus$ s54	s120 = s92 $\oplus$ s56
s55 = s21 $\oplus$ s29	s88 = s43 $\oplus$ s56	s121 = s90 $\oplus$ s36
s56 = s22 $\oplus$ s30	s89 = s55 $\oplus$ s30	s122 = s69 $\oplus$ s90
s57 = s7 $\oplus$ s31	s90 = s57 $\oplus$ s0	s123 = s74 $\oplus$ s2
s58 = s23 $\oplus$ s31	s91 = s57 $\oplus$ s3	s124 = s76 $\oplus$ s91
s59 = s33 $\oplus$ s14	s92 = s57 $\oplus$ s15	s125 = s82 $\oplus$ s91
s60 = s35 $\oplus$ s17	s93 = s34 $\oplus$ s58	s126 = s85 $\oplus$ s33
s61 = s39 $\oplus$ s10	s94 = s44 $\oplus$ s58	s127 = s86 $\oplus$ s59
s62 = s41 $\oplus$ s5	s95 = s48 $\oplus$ s58	s128 = s88 $\oplus$ s64
s63 = s35 $\oplus$ s40	s96 = s53 $\oplus$ s58	
s64 = s42 $\oplus$ s7	s97 = s68 $\oplus$ s35	

Output signals (output bit j is carried by the listed signal, $j = 0, \dots, 31$):

[97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128]

B Circuit mixcolumns_92gates_depth4 (92 gates, depth 4)

Signals $s_0, \dots, s_{31}$ are the inputs; gate k produces s_{32+k} . Canonical gate-list SHA-256 (see Section 4):

e79bfccf5322fe8933242b47b53e94e73163615c70d5c45c9721e27ac86fd97d

s32 = s10 $\oplus$ s26	s63 = s32 $\oplus$ s56	s94 = s35 $\oplus$ s77
s33 = s12 $\oplus$ s20	s64 = s36 $\oplus$ s56	s95 = s15 $\oplus$ s61
s34 = s14 $\oplus$ s30	s65 = s37 $\oplus$ s56	s96 = s69 $\oplus$ s74
s35 = s6 $\oplus$ s14	s66 = s1 $\oplus$ s48	s97 = s65 $\oplus$ s72
s36 = s11 $\oplus$ s28	s67 = s50 $\oplus$ s53	s98 = s38 $\oplus$ s60
s37 = s8 $\oplus$ s16	s68 = s35 $\oplus$ s56	s99 = s70 $\oplus$ s71
s38 = s7 $\oplus$ s31	s69 = s3 $\oplus$ s38	s100 = s57 $\oplus$ s46
s39 = s17 $\oplus$ s25	s70 = s51 $\oplus$ s54	s101 = s39 $\oplus$ s78
s40 = s13 $\oplus$ s21	s71 = s19 $\oplus$ s48	s102 = s39 $\oplus$ s65
s41 = s2 $\oplus$ s26	s72 = s24 $\oplus$ s38	s103 = s69 $\oplus$ s50
s42 = s5 $\oplus$ s29	s73 = s39 $\oplus$ s41	s104 = s63 $\oplus$ s49
s43 = s16 $\oplus$ s24	s74 = s41 $\oplus$ s55	s105 = s64 $\oplus$ s50
s44 = s9 $\oplus$ s25	s75 = s34 $\oplus$ s38	s106 = s64 $\oplus$ s69
s45 = s18 $\oplus$ s26	s76 = s43 $\oplus$ s48	s107 = s23 $\oplus$ s75
s46 = s0 $\oplus$ s17	s77 = s22 $\oplus$ s42	s108 = s71 $\oplus$ s80
s47 = s1 $\oplus$ s9	s78 = s41 $\oplus$ s44	s109 = s46 $\oplus$ s72
s48 = s23 $\oplus$ s31	s79 = s10 $\oplus$ s47	s110 = s102 $\oplus$ s66
s49 = s3 $\oplus$ s27	s80 = s45 $\oplus$ s49	s111 = s18 $\oplus$ s101
s50 = s4 $\oplus$ s20	s81 = s21 $\oplus$ s33	s112 = s89 $\oplus$ s78
s51 = s12 $\oplus$ s27	s82 = s67 $\oplus$ s81	s113 = s63 $\oplus$ s108
s52 = s22 $\oplus$ s30	s83 = s57 $\oplus$ s66	s114 = s108 $\oplus$ s55
s53 = s13 $\oplus$ s29	s84 = s58 $\oplus$ s77	s115 = s93 $\oplus$ s65
s54 = s4 $\oplus$ s28	s85 = s42 $\oplus$ s81	s116 = s47 $\oplus$ s109
s55 = s11 $\oplus$ s19	s86 = s59 $\oplus$ s67	s117 = s52 $\oplus$ s107
s56 = s15 $\oplus$ s31	s87 = s59 $\oplus$ s40	s118 = s96 $\oplus$ s104
s57 = s43 $\oplus$ s44	s88 = s10 $\oplus$ s73	s119 = s33 $\oplus$ s106
s58 = s34 $\oplus$ s53	s89 = s45 $\oplus$ s79	s120 = s71 $\oplus$ s105
s59 = s5 $\oplus$ s54	s90 = s23 $\oplus$ s68	s121 = s103 $\oplus$ s51
s60 = s0 $\oplus$ s37	s91 = s62 $\oplus$ s52	s122 = s97 $\oplus$ s100
s61 = s38 $\oplus$ s52	s92 = s58 $\oplus$ s62	s123 = s90 $\oplus$ s75
s62 = s6 $\oplus$ s40	s93 = s60 $\oplus$ s76	

Output signals (output bit j is carried by the listed signal, $j = 0, \dots, 31$):

[97, 122, 89, 118, 119, 82, 84, 90, 115, 110, 111, 113, 120, 85, 91, 117, 93, 83, 88, 114, 99, 86, 92, 95, 98, 116, 112, 96, 121, 87, 94, 123]

C Circuit mixcolumns_89gates_depth5 (89 gates, depth 5)

Signals $s_0, \dots, s_{31}$ are the inputs; gate k produces s_{32+k} . Canonical gate-list SHA-256 (see Section 4):

c3b972af282d25e8313a510f5cf840472401c6268bd28cda889593fc3379f064

s32 = s9 $\oplus$ s17	s49 = s6 $\oplus$ s14	s66 = s36 $\oplus$ s41
s33 = s19 $\oplus$ s27	s50 = s19 $\oplus$ s20	s67 = s41 $\oplus$ s44
s34 = s1 $\oplus$ s25	s51 = s8 $\oplus$ s16	s68 = s34 $\oplus$ s53
s35 = s21 $\oplus$ s29	s52 = s7 $\oplus$ s31	s69 = s24 $\oplus$ s45
s36 = s12 $\oplus$ s28	s53 = s2 $\oplus$ s18	s70 = s40 $\oplus$ s47
s37 = s22 $\oplus$ s30	s54 = s30 $\oplus$ s35	s71 = s31 $\oplus$ s49
s38 = s3 $\oplus$ s27	s55 = s21 $\oplus$ s39	s72 = s26 $\oplus$ s34
s39 = s4 $\oplus$ s28	s56 = s5 $\oplus$ s46	s73 = s32 $\oplus$ s48
s40 = s9 $\oplus$ s25	s57 = s14 $\oplus$ s42	s74 = s44 $\oplus$ s45
s41 = s13 $\oplus$ s29	s58 = s15 $\oplus$ s37	s75 = s40 $\oplus$ s51
s42 = s5 $\oplus$ s13	s59 = s11 $\oplus$ s33	s76 = s18 $\oplus$ s73
s43 = s15 $\oplus$ s23	s60 = s11 $\oplus$ s43	s77 = s10 $\oplus$ s68
s44 = s6 $\oplus$ s22	s61 = s0 $\oplus$ s52	s78 = s55 $\oplus$ s66
s45 = s7 $\oplus$ s23	s62 = s33 $\oplus$ s45	s79 = s37 $\oplus$ s57
s46 = s12 $\oplus$ s20	s63 = s38 $\oplus$ s52	s80 = s35 $\oplus$ s56
s47 = s10 $\oplus$ s18	s64 = s43 $\oplus$ s51	s81 = s56 $\oplus$ s66
s48 = s2 $\oplus$ s26	s65 = s8 $\oplus$ s45	s82 = s55 $\oplus$ s42

$s_{83} = s_{70} \oplus s_{72}$
 $s_{84} = s_{43} \oplus s_{71}$
 $s_{85} = s_{54} \oplus s_{67}$
 $s_{86} = s_{54} \oplus s_{49}$
 $s_{87} = s_{17} \oplus s_{64}$
 $s_{88} = s_{57} \oplus s_{67}$
 $s_{89} = s_{60} \oplus s_{47}$
 $s_{90} = s_{62} \oplus s_{53}$
 $s_{91} = s_{71} \oplus s_{74}$
 $s_{92} = s_{24} \oplus s_{61}$
 $s_{93} = s_{38} \oplus s_{62}$
 $s_{94} = s_{58} \oplus s_{52}$
 $s_{95} = s_{59} \oplus s_{63}$

$s_{96} = s_{64} \oplus s_{69}$
 $s_{97} = s_{61} \oplus s_{51}$
 $s_{98} = s_0 \oplus s_{65}$
 $s_{99} = s_4 \oplus s_{63}$
 $s_{100} = s_{58} \oplus s_{74}$
 $s_{101} = s_{60} \oplus s_{50}$
 $s_{102} = s_{39} \oplus s_{101}$
 $s_{103} = s_{34} \oplus s_{87}$
 $s_{104} = s_{59} \oplus s_{89}$
 $s_{105} = s_{76} \oplus s_{70}$
 $s_{106} = s_{92} \oplus s_{65}$
 $s_{107} = s_{96} \oplus s_{98}$
 $s_{108} = s_{36} \oplus s_{93}$

$s_{109} = s_{95} \oplus s_{48}$
 $s_{110} = s_1 \oplus s_{92}$
 $s_{111} = s_{99} \oplus s_{46}$
 $s_{112} = s_{98} \oplus s_{75}$
 $s_{113} = s_{89} \oplus s_{90}$
 $s_{114} = s_{108} \oplus s_{99}$
 $s_{115} = s_{110} \oplus s_{112}$
 $s_{116} = s_3 \oplus s_{104}$
 $s_{117} = s_{90} \oplus s_{109}$
 $s_{118} = s_{108} \oplus s_{101}$
 $s_{119} = s_{87} \oplus s_{112}$
 $s_{120} = s_{32} \oplus s_{110}$

Output signals (output bit j is carried by the listed signal, $j = 0, \dots, 31$):

$[96, 119, 83, 113, 118, 78, 79, 84, 107, 103, 76, 116, 102, 80, 85, 91, 106, 115, 105, 117, 114, 81, 86,$
 $94, 97, 120, 77, 109, 111, 82, 88, 100]$